LAB: Password cracking

Password Cracking en DVWA

Estamos simulando un ataque en el que un intruso ya ha logrado comprometer la base de datos de una aplicación web. El objetivo es demostrar que, incluso si las contraseñas no se guardan en texto claro (gracias al hashing), aún pueden ser vulnerables si los algoritmos de hashing son débiles o las contraseñas son sencillas.

Password Cracking e Vulnerabilità dei Database

Nel mondo della cybersecurity, non esiste un unico strumento perfetto; la scelta dipende dalle risorse a disposizione (RAM, GPU) e dal tipo di hash. In questo progetto abbiamo esplorato tre strade:

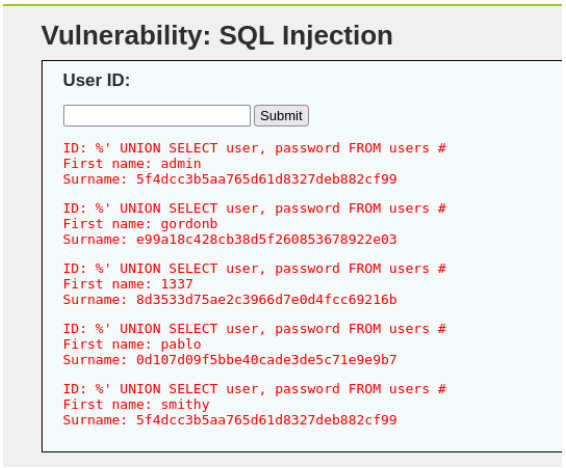
---

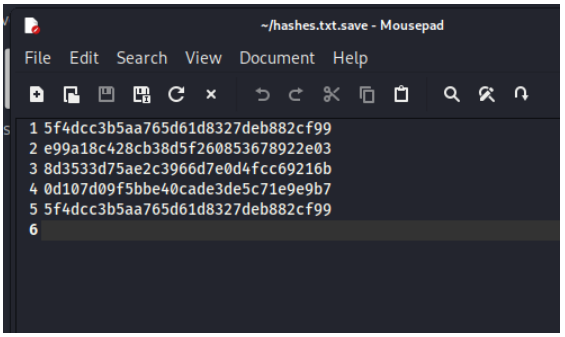
1. John the Ripper (La scelta principale)

John the Ripper è stato il nostro strumento di successo. È un software "CPU-based", il che significa che utilizza la potenza del processore centrale.

- **Perché lo abbiamo usato:** È estremamente leggero e intelligente. È in grado di rilevare automaticamente il tipo di hash (nel nostro caso MD5) e di adattarsi alle risorse limitate di una macchina virtuale.

- **Come ha funzionato:** Abbiamo eseguito un **Attacco di Dizionario**. John prende ogni parola dal suo file di testo (`password.lst`), calcola il suo valore MD5 e lo confronta con i nostri hash estratti da DVWA.
- **Risultato:** È riuscito a rompere tutte le password in pochi secondi perché le chiavi erano semplici e corte.






```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-md5 hashes.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password         (?)
password         (?)
abc123           (?)
letmein          (?)
Proceeding with incremental:ASCII
charley          (?)
5g 0:00:00:00 DONE 3/3 (2026-04-15 06:52) 15.62g/s 557343p/s 557343c/s 562143C/s stevy13..candake
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

┌──(kali㉿kali)-[~]
└─$ john --show --format=raw-md5 hashes.txt
?:password
?:abc123
?:charley
?:letmein
?:password

5 password hashes cracked, 0 left

┌──(kali㉿kali)-[~]
└─$
```

Usuario (de DVWA)	Hash MD5	Contraseña Crackeada
admin	`5f4dcc3b5aa765d61d8327deb882cf99`	password

gordonb	`e99a18c428cb38d5f260853678922e03`	abc123
1337	`8d3533d75ae2c3966d7e0d4fcc69216b`	charley
pablo	`0d107d09f5bbe40cade3de5c71e9e9b7`	letmein
smithy	`5f4dcc3b5aa765d61d8327deb882cf99`	password

Comandi usati:

```
john --format=raw-md5 hashes.txt :
```

Questo è il comando principale per l'attacco di cracking. Ecco cosa significa ogni parte:

- `john`: Avvia il programma **John the Ripper**, uno dei cracker di password più famosi al mondo.
- `-format=raw-md5`: Questa è l'istruzione più importante. Dice al programma esattamente che tipo di "lingua" stiamo parlando.
  - Gli hash MD5 possono avere diversi formati.
  - `raw-md5` indica che sono hash "puri", ovvero semplici stringhe MD5 senza protezioni aggiuntive (come il "salt"). Specificandolo, John non deve perdere tempo a indovinare il formato e va dritto al punto.
- `hashes.txt`: È il file che contiene la lista di codici (gli hash) che abbiamo estratto dal database di DVWA.

```
mv hashes.txt.save hashes.txt:
```

Cosa fa: Questo comando serve a **rinominare** un file. La parola `mv` sta per *move* (muovere). In Linux, muovere un file nello stesso posto ma con un nome diverso equivale a rinominarlo.

- `hashes.txt.save`: È il nome del file originale (quello che avevi creato con l'editor nano).
- `hashes.txt`: È il nuovo nome che volevi dare al file.

---

2. Hashcat (Potenza Bruta e GPU)

Hashcat è considerato il "re" dei cracker, ma nel nostro test ha riscontrato un errore di memoria.

- **Il problema della memoria:** Hashcat è progettato per caricare dizionari massicci (come **RockYou.txt**, che ha 14 milioni di parole) direttamente nella memoria della scheda video (GPU) o nella RAM. Poiché la nostra Macchina Virtuale aveva poca RAM assegnata, il sistema ha bloccato il processo per protezione.
- **Perché è comunque importante:** In un ambiente professionale (non virtualizzato), Hashcat è infinitamente più veloce di John perché può usare la potenza di calcolo parallelo delle schede video. Può testare miliardi di combinazioni al secondo.
- **Lezione appresa:** Anche se non ha completato il lavoro a causa dell'hardware, abbiamo configurato correttamente i parametri (`m 0` per MD5) e identificato che le password non avevano un "Salt", facilitando teoricamente il lavoro dello strumento.

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

/usr/share/wordlists/rockyou.txt: No such file or directory

Started: Wed Apr 15 06:58:31 2026
Stopped: Wed Apr 15 06:58:31 2026

┌──(kali㉿kali)-[~]
└─$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/wordlists/rockyou.txt
/usr/share/wordlists/rockyou.txt

┌──(kali㉿kali)-[~]
└─$ hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) -
 Platform #1 [The pocl project]
==================================================================================================
* Device #01: cpu-haswell-11th Gen Intel(R) Core(TM) i7-1195G7 @ 2.90GHz, 738/1477 MB (256 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

* Device #1: Not enough allocatable device memory or free host memory for mapping.

Started: Wed Apr 15 07:04:12 2026
Stopped: Wed Apr 15 07:05:31 2026
```

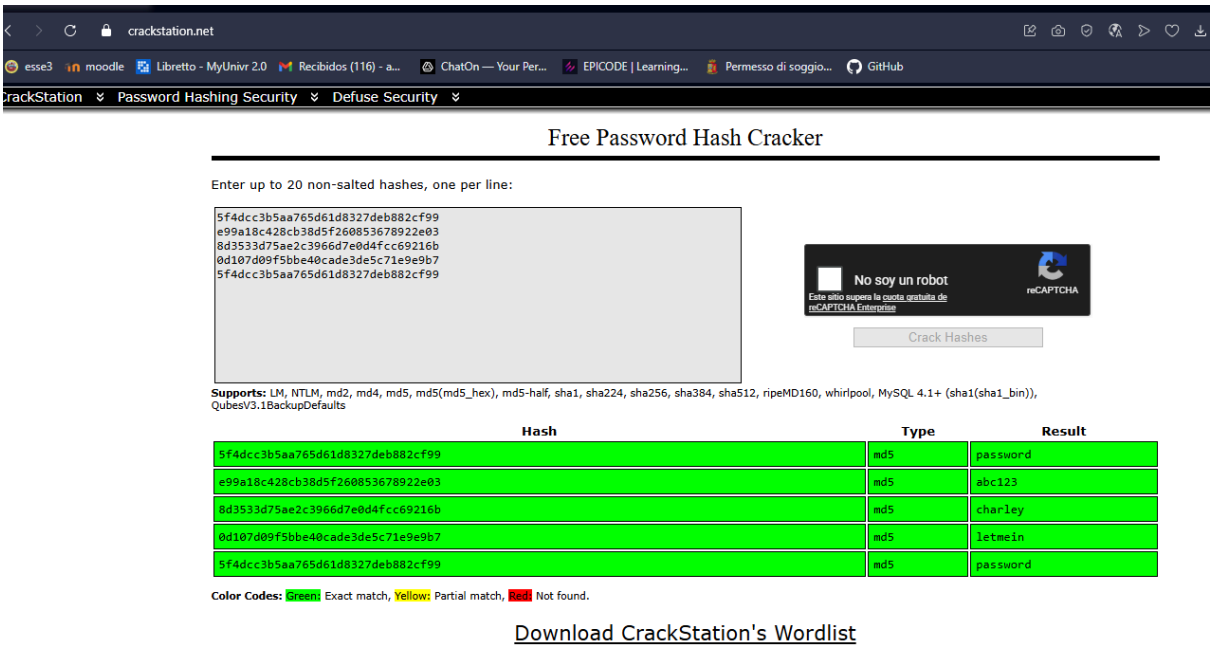
```
Session  Actions  Edit  View  Help
ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

* Device #1: Not enough allocatable device memory or free host memory for mapping.

Started: Wed Apr 15 07:04:12 2026
Stopped: Wed Apr 15 07:05:31 2026

┌──(kali㉿kali)-[~]
└─$ hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt -O -w 1 --force
hashcat (v7.1.2) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) -
 Platform #1 [The pocl project]
==================================================================================================
* Device #01: cpu-haswell-11th Gen Intel(R) Core(TM) i7-1195G7 @ 2.90GHz, 738/1477 MB (256 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Optimized-Kernel
* Zero-Byte
* Precompute-Init
* Meet-In-The-Middle
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

Watchdog: Temperature abort trigger set to 90c

* Device #1: Not enough allocatable device memory or free host memory for mapping.

Started: Wed Apr 15 07:07:06 2026
Stopped: Wed Apr 15 07:07:26 2026

┌──(kali㉿kali)-[~]
└─$
```

3. Rainbow Tables / CrackStation (Metodo Online)

Questo è il metodo più veloce in assoluto, ma non è un vero "cracking" in tempo reale.

- **Cos'è una Rainbow Table:** È un database gigantesco dove qualcuno ha già calcolato e salvato i risultati. Invece di calcolare l'hash di "password", il sito cerca semplicemente nel suo indice: *"Chi ha questo hash?"* e trova la risposta istantaneamente.
- **Vantaggio:** È immediato e non consuma risorse della tua batteria o del tuo computer.
- **Svantaggio e Etica:** Non può rompere password nuove o complesse che non sono mai state viste prima. Inoltre, **mai**

usare questo metodo con dati reali di un cliente, perché carichi la loro password su un server pubblico di terze parti.



Sintesi:

Tecnica	Strumento	Punto di Forza	Risultato nel Progetto
Dizionario (CPU)	John the Ripper	Versatilità e leggerezza	**Successo** (Password trovate)
Forza Bruta (GPU)	Hashcat	Velocità estrema	**Errore Memory** (Limite HW)
Lookup Tables	CrackStation	Istantaneo	**Successo** (Conferma dati)

Conclusione:

Il progetto dimostra che anche con hardware limitato (VM), un attaccante può recuperare le credenziali se il database utilizza algoritmi obsoleti come MD5 e password deboli. La difesa migliore rimane l'uso di algoritmi moderni e l'assegnazione di più risorse di sicurezza lato server.